

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ

<Họ và tên sinh viên>

Mã sinh viên: <Mã sinh viên>

XÂY DỰNG HỆ THỐNG TUYỂN DỤNG JOBCONNECT MVP

VỚI QUY TRÌNH ATS-LITE VÀ SO KHỚP CV-JD CÓ GIẢI
THÍCH

KHÓA LUẬN TỐT NGHIỆP ĐẠI HỌC HỆ CHÍNH QUY

Ngành: <Tên khoa/ngành nếu cần>

Cán bộ hướng dẫn: <Giảng viên hướng dẫn>

HÀ NỘI - 2026

Tóm tắt

Khóa luận trình bày quá trình phân tích, thiết kế và cài đặt hệ thống JobConnect MVP, một nền tảng tuyển dụng theo mô hình marketplace dùng chung một vùng dữ liệu công khai giữa ứng viên và nhà tuyển dụng. Hệ thống tập trung vào ba năng lực chính: chuẩn hóa dữ liệu CV/JD từ tài liệu phi cấu trúc, tìm kiếm và so khớp hai chiều giữa hồ sơ ứng viên và tin tuyển dụng, và hỗ trợ luồng ATS-lite gồm ứng tuyển, lời mời, thông báo và nhật ký nghiệp vụ. Báo cáo sử dụng tài liệu đặc tả trong repository làm nguồn chính, gồm yêu cầu sản phẩm, kiến trúc backend, thiết kế dữ liệu, hợp đồng API và ma trận hiện trạng triển khai.

Về mặt kỹ thuật, JobConnect được xây dựng xoay quanh backend FastAPI viết bằng Python, cơ sở dữ liệu PostgreSQL, vector semantic thông qua pgvector, xác thực JWT, các module nghiệp vụ tách theo miền và các biên tích hợp cho lưu trữ file, LLM parser, embedding provider, reranker và email. Pipeline xử lý tài liệu đi từ upload file, lưu metadata, tạo parse job, tiền xử lý văn bản, chuẩn hóa kỹ năng, trích xuất cấu trúc bằng parser, sinh embedding, rồi đưa dữ liệu đã review vào public pool khi người dùng kích hoạt hồ sơ hoặc publish job.

Đóng góp chính của khóa luận là mô hình hóa rõ bài toán tuyển dụng từ CV/JD phi cấu trúc thành một hệ thống production MVP có hợp đồng API, schema dữ liệu, rule matching, luồng lifecycle và cơ chế giải thích kết quả. Báo cáo cũng chỉ ra các phần đã triển khai, các phần còn ở trạng thái partial và các hướng phát triển tiếp theo như hoàn thiện parse review payload, email provider, frontend production integration và benchmark có nhãn.

Lời cam đoan

Tôi xin cam đoan nội dung trong khóa luận này được xây dựng dựa trên quá trình nghiên cứu, phân tích và tổng hợp từ source code, tài liệu yêu cầu và tài liệu thiết kế của dự án JobConnect trong repository. Các tài liệu tham khảo, benchmark bên ngoài và tài liệu nội bộ được chỉ dẫn trong danh mục tài liệu tham khảo. Những thông tin cá nhân chưa được cung cấp trong quá trình soạn thảo được giữ ở dạng placeholder để bổ sung sau.

Lời cảm ơn

Tôi xin gửi lời cảm ơn tới <Giảng viên hướng dẫn> đã định hướng và góp ý trong quá trình thực hiện đề tài. Tôi cũng xin cảm ơn các thầy cô trong <Tên khoa/ngành nếu cần> đã cung cấp nền tảng kiến thức về phát triển phần mềm, cơ sở dữ liệu, hệ thống web và trí tuệ nhân tạo ứng dụng. Những kiến thức này là cơ sở để phân tích và xây dựng hệ thống JobConnect MVP theo hướng có khả năng vận hành thực tế.

Contents

Tóm tắt	i
Lời cam đoan	ii
Lời cảm ơn	iii
Danh mục bảng	ix
Danh mục hình	x
Danh mục ký hiệu và chữ viết tắt	xi
Mở đầu	1
Lý do chọn đề tài	1
Mục tiêu của khóa luận	2
Đối tượng và phạm vi nghiên cứu	3
Phương pháp thực hiện	3
Đóng góp của khóa luận	4
Bố cục báo cáo	5
1 Giới thiệu chung và cơ sở bài toán	6

1.1	Bối cảnh tuyển dụng số	6
1.2	Nhóm người dùng và nhu cầu	7
1.3	Phát biểu bài toán	8
1.4	Phạm vi MVP và các non-goals	9
1.5	Các khái niệm nền tảng	10
1.5.1	ATS-lite workflow	10
1.5.2	Structured parsing	10
1.5.3	Embedding và pgvector	10
1.5.4	Explainable matching	11
1.6	Yêu cầu trạng thái và enum canonical	11
1.7	Liên hệ với báo cáo cũ và định hướng viết lại	12
2	Mô hình và kỹ thuật nền tảng	13
2.1	Kiến trúc web API với FastAPI	13
2.2	Pydantic, schema validation và error envelope	14
2.3	PostgreSQL và pgvector	15
2.4	Pipeline xử lý tài liệu CV/JD	16
2.4.1	Preprocessing	17
2.4.2	Skill normalization	17
2.4.3	LLM structured parsing	17
2.5	Embedding provider và semantic search	18
2.6	Matching pipeline	18
2.6.1	Hard filters	19
2.6.2	Scoring deterministic	19
2.6.3	Reranking và reasoning	20

2.7	Authentication, authorization và auditability	20
3	Phân tích và thiết kế hệ thống đề xuất	22
3.1	Tổng quan kiến trúc JobConnect MVP	22
3.2	Thiết kế module nghiệp vụ	23
3.3	Thiết kế dữ liệu	24
3.3.1	Identity và organization	25
3.3.2	Resume và job	25
3.3.3	Documents, parse jobs và embeddings	25
3.3.4	Applications, invites, notifications và audit	26
3.4	Thiết kế API	26
3.5	Thiết kế luồng runtime	27
3.5.1	Candidate CV flow	27
3.5.2	Recruiter JD flow	28
3.5.3	Matching flow	28
3.5.4	Application và invite flow	28
3.6	Thiết kế bảo mật và quyền truy cập	28
3.7	Thiết kế observability	29
3.8	Điểm sáng tạo và khác biệt của mô hình đề xuất	29
4	Cài đặt thực nghiệm và đánh giá	31
4.1	Môi trường và công nghệ triển khai	31
4.2	Cài đặt backend runtime	32
4.3	Cài đặt cơ sở dữ liệu	33
4.4	Cài đặt document upload và parse worker	34
4.5	Cài đặt matching và semantic search	35

4.6	Cài đặt auth, authorization và lifecycle	36
4.7	Cài đặt notifications và audit	36
4.8	Bộ kiểm thử hiện có	37
4.9	Đối chiếu hiện trạng với target contract	38
4.10	Đánh giá theo yêu cầu sản phẩm	39
4.10.1	Điểm đáp ứng tốt	39
4.10.2	Điểm còn partial	39
4.10.3	Rủi ro kỹ thuật	40
4.11	Kịch bản thực nghiệm đề xuất	40
4.12	Nhận xét chương	41
Kết luận		42
	Kết quả đạt được	42
	Hạn chế	43
	Hướng phát triển	44
	Kết luận chung	45
Tài liệu tham khảo		46
A Phụ lục API, schema và traceability		48
A.1	Tóm tắt API contract	48
A.2	Entity summary	51
A.3	Matching formula và hard filter details	53
A.4	Source traceability	54
A.5	Checklist dùng lại khi tiếp tục phát triển báo cáo	55

List of Tables

2	Nguồn tài liệu chính dùng trong báo cáo	4
1.1	Vai trò người dùng và năng lực chính	7
1.2	MVP scope và non-goals	9
1.3	Enum canonical trong JobConnect MVP	11
2.1	Nhóm bảng chính trong schema JobConnect	16
2.2	Cấu hình embedding provider	18
2.3	Hard filters trong matching	19
3.1	Mapping thư mục backend sang trách nhiệm kiến trúc	22
3.2	Feature modules và API namespace liên quan	24
3.3	Production API namespaces	26
4.1	Dependency backend cụ thể trong dự án	31
4.2	Một số invariant database quan trọng	34
4.3	Các bước matching trong runtime	35
4.4	Các nhóm test hiện có trong backend	37
4.5	Tóm tắt đối chiếu hiện trạng triển khai	38
A.1	API endpoint summary theo nhóm chức năng	48

A.2	Tóm tắt entity và field chính	52
A.3	Score component trong matching	53
A.4	Reasoning content tối thiểu	53
A.5	Traceability từ nội dung báo cáo về source docs	54
A.6	Checklist kiểm tra nội dung báo cáo	55

List of Figures

2.1	Kiến trúc runtime tổng quát của JobConnect	14
2.2	Pipeline xử lý CV/JD từ file đến public pool	16
3.1	Quan hệ logic giữa các nhóm entity	26
3.2	Luồng candidate từ CV đến ứng tuyển	27

Danh mục ký hiệu và chữ viết tắt

Ký hiệu	Ý nghĩa
ATS	Applicant Tracking System, hệ thống quản lý quy trình ứng tuyển.
CV	Curriculum Vitae, hồ sơ ứng viên.
JD	Job Description, mô tả công việc hoặc tin tuyển dụng.
MVP	Minimum Viable Product, phiên bản sản phẩm khả dụng tối thiểu.
JWT	JSON Web Token, token dùng cho phiên đăng nhập API.
LLM	Large Language Model, mô hình ngôn ngữ lớn.
API	Application Programming Interface.
OpenAPI	Đặc tả machine-readable cho hợp đồng API.
pgvector	Extension PostgreSQL dùng lưu và truy vấn vector embedding.
RAG	Retrieval-Augmented Generation, kiến trúc kết hợp truy hồi và sinh nội dung.

Mở đầu

Lý do chọn đề tài

Trong nhiều doanh nghiệp, tuyển dụng không chỉ là hoạt động đăng tin và nhận hồ sơ, mà là một chuỗi xử lý dữ liệu có nhiều trạng thái: ứng viên tạo hồ sơ, nhà tuyển dụng công bố nhu cầu, hệ thống phân tích CV/JD, hai phía tìm kiếm lẫn nhau, kết quả phù hợp được giải thích, sau đó mới phát sinh ứng tuyển hoặc lời mời. Nếu chuỗi này chỉ được xử lý bằng bảng tính, email thủ công hoặc các form nhập liệu đơn giản, chất lượng matching phụ thuộc rất lớn vào kinh nghiệm cá nhân của người vận hành. Điều này tạo ra ba vấn đề chính: dữ liệu không đồng nhất, khó kiểm soát trạng thái nghiệp vụ và khó giải thích vì sao một ứng viên phù hợp hoặc không phù hợp với một vị trí.

Dự án JobConnect MVP được xây dựng để giải quyết nhóm vấn đề trên ở mức sản phẩm khả dụng tối thiểu. Theo đặc tả sản phẩm [1], hệ thống không còn là một prototype chạy riêng chức năng matching, mà là một recruiting marketplace có luồng ATS-lite. Ứng viên có thể upload một hoặc nhiều CV, review dữ liệu đã parse và kích hoạt resume vào public pool. Nhà tuyển dụng có thể tạo tổ chức, tạo hoặc upload job description, publish job vào public pool, chạy matching từ job sang active resumes, gửi lời mời và quản lý ứng tuyển. Admin theo dõi người dùng, nội dung, parse jobs, matching activity, notifications và audit events.

Điểm đáng chú ý của bài toán là dữ liệu đầu vào thường là văn bản phi cấu trúc. CV và JD có thể viết bằng tiếng Việt, tiếng Anh hoặc trộn cả hai. Một kỹ năng như “ReactJS”, “React.js” và “React” cần được chuẩn hóa về cùng một nhãn; một bằng cấp như “Master of Science in CS” cần ánh xạ về enum `thac_si`; một vị trí “Senior Developer” cần ánh

xạ về **senior**. Nếu chỉ lưu văn bản thô và tìm kiếm bằng từ khóa, hệ thống khó tận dụng thông tin ngữ nghĩa. Nếu chỉ dùng LLM không có schema và rule, hệ thống dễ sinh nhảm không hợp lệ, khó kiểm soát và khó debug. Vì vậy JobConnect chọn hướng kết hợp: lưu dữ liệu nghiệp vụ có cấu trúc trong PostgreSQL, lưu embedding semantic bằng pgvector, dùng rule hard filter rõ ràng, dùng scoring deterministic và chỉ cho phép LLM reasoning khi reasoning bám vào score breakdown [6].

Mục tiêu của khóa luận

Khóa luận này đặt mục tiêu mô tả và đánh giá quá trình xây dựng JobConnect MVP ở mức có thể chuyển thành sản phẩm thực tế. Báo cáo không chỉ nói rằng dự án “dùng Python”, mà chỉ rõ backend đang dùng FastAPI 0.115.6, Pydantic 2.11.7, psycopg 3.2.3, PostgreSQL, pgvector, JWT, Python multipart upload, pypdf cho trích xuất PDF và sentence-transformers cho local rerank. Báo cáo cũng chỉ rõ các module runtime nằm dưới `backend/src/jobconnect`, gồm `auth`, `users`, `organizations`, `jobs`, `resumes`, `documents`, `matching`, `applications`, `invites`, `notifications`, `admin` và `system`.

Các mục tiêu cụ thể gồm:

- Phân tích yêu cầu sản phẩm của một shared-pool recruiting marketplace với ba vai trò `candidate`, `recruiter` và `admin`.
- Mô hình hóa dữ liệu CV/JD, trạng thái `resume/job`, `parse job`, `application`, `invite`, `notification` và `audit log` theo schema PostgreSQL cụ thể.
- Thiết kế pipeline xử lý tài liệu từ upload file đến dữ liệu chuẩn hóa và embedding.
- Trình bày thuật toán `matching` hai chiều với `hard filters`, công thức `scoring`, `tie-break deterministic`, `optional rerank` và `reasoning` có căn cứ.
- Đối chiếu hiện trạng triển khai với target API contract để chỉ ra phần đã aligned, phần `partial` và các rủi ro còn lại.

Đối tượng và phạm vi nghiên cứu

Đối tượng nghiên cứu của khóa luận là hệ thống backend JobConnect MVP và các tài liệu thiết kế đi kèm trong repository. Phạm vi tập trung vào backend production API, data model, parse/matching pipeline và workflow nghiệp vụ. Frontend runtime chưa phải phần code hoạt động trong repository; các tài liệu frontend hiện tại chỉ được xem là planning notes, không phải source of truth cho triển khai hiện tại [12]. Do đó, báo cáo có nhắc đến các màn hình như Job Market, Talent Market hoặc parse review page ở mức luồng sử dụng và yêu cầu dữ liệu, nhưng không trình bày như một frontend production đã hoàn thiện.

Theo yêu cầu sản phẩm [1], MVP có các phần nằm ngoài phạm vi: tenant/company isolation, billing, quota, scheduling phỏng vấn đầy đủ, offer approval, onboarding, CRM campaign, destructive admin moderation, salary/language/visa/notice period như hard filter, knowledge graph cho job titles/skills, BM25 hybrid retrieval và quyết định reject/advance hoàn toàn tự động bằng AI. Việc loại bỏ các scope này giúp hệ thống tập trung vào lõi: public pool, structured profiles, two-way matching, application lifecycle, notification và auditability.

Phương pháp thực hiện

Phương pháp thực hiện gồm bốn bước. Bước thứ nhất là đọc và chuẩn hóa yêu cầu từ docs/REQUIREMENTS.md. Đây là tài liệu xác định sản phẩm cần xây gì, vì sao, dữ liệu nào là canonical, trạng thái nào hợp lệ, matching formula ra sao và các acceptance criteria chính. Bước thứ hai là phân tích các tài liệu HLD/LLD để hiểu kiến trúc backend, ingestion, data storage, matching, runtime flow, security và API contract. Bước thứ ba là đối chiếu với code layout và ma trận hiện trạng triển khai để xác định những phần đã có trong runtime, ví dụ API namespaces dưới /api/*, migration 001_production_mvp.sql, provider boundaries cho LLM/embedding/storage/rerank và các test hiện có. Bước cuối cùng là tổng hợp thành một báo cáo LaTeX có bố cục khóa luận, có bảng, công thức, phụ lục và traceability từ nội dung báo cáo về nguồn tài liệu.

Table 2: Nguồn tài liệu chính dùng trong báo cáo

Nguồn	Vai trò trong báo cáo
docs/REQUIREMENTS.md	Định nghĩa product goal, scope, enum, user flows, data model, matching formula và API namespaces.
docs/backend/HLD/*	Mô tả kiến trúc, ingestion, data storage, matching/search, runtime flow, security, notification và audit.
docs/backend/LLD/*	Chi tiết schema PostgreSQL/pgvector, API contract và implementation matrix hiện tại.
README.md	Xác nhận baseline runtime, layout backend, cách setup Docker cho backend và tình trạng frontend.
backend/requirements.txt	Xác nhận dependency cụ thể như FastAPI, Pydantic, psychopg, pypdf và sentence-transformers.
report/knowledge/01-04	Quy định bố cục, giọng viết, mapping nội dung cũ và rule sử dụng knowledge pack.

Đóng góp của khóa luận

Khóa luận có ba đóng góp chính. Thứ nhất, báo cáo hệ thống hóa bài toán tuyển dụng thành một mô hình dữ liệu và workflow rõ ràng thay vì chỉ mô tả matching ở mức thuật toán. Điều này thể hiện qua việc phân tách resume/job/document/parse job/application/invite/notification/audit log thành các thực thể riêng, mỗi thực thể có trạng thái và invariant cụ thể.

Thứ hai, khóa luận mô tả một pipeline matching có thể giải thích. Các cặp JD-CV trước hết phải qua hard filters về `job_type`, `location`, `seniority`, `education` và `certifications`. Sau đó score được tính bằng công thức explicit:

$$\begin{aligned}
 final_score = & 0.35 \times title_sim + 0.35 \times skills_sim \\
 & + 0.20 \times req_exp_sim + 0.10 \times req_summary_sim \\
 & + bonus_exact_skill - penalty_missing_required
 \end{aligned} \tag{1}$$

Trong đó:

$$skills_sim = 0.6 \times semantic_skills + 0.4 \times exact_overlap_ratio \quad (2)$$

Thứ ba, khóa luận chỉ ra ranh giới giữa target architecture và hiện trạng implementation. Ví dụ, upload documents đã có adapter storage local và parse jobs, nhưng parse job detail vẫn còn thin so với target review payload; notification rows đã có trạng thái email queued, nhưng email provider thực sự là future work; frontend runtime chưa active, nên báo cáo không tuyên bố đã có production frontend.

Bố cục báo cáo

Báo cáo được tổ chức theo bốn chương nội dung. Chương 1 giới thiệu bối cảnh, bài toán, mục tiêu và các khái niệm nền tảng. Chương 2 trình bày các kỹ thuật và mô hình nền tảng được sử dụng, gồm FastAPI, PostgreSQL, pgvector, ingestion pipeline, LLM parser, embedding provider và matching pipeline. Chương 3 phân tích và thiết kế hệ thống JobConnect MVP, từ kiến trúc module, data model đến API/runtime flows. Chương 4 trình bày cài đặt, môi trường, kiểm thử, ma trận hiện trạng triển khai và đánh giá. Phần Kết luận tổng hợp kết quả, hạn chế và hướng phát triển.

CHAPTER 1

Giới thiệu chung và cơ sở bài toán

1.1 Bối cảnh tuyển dụng số

Tuyển dụng hiện đại là một bài toán dữ liệu nhiều phía. Ứng viên không chỉ cung cấp một bộ thông tin định danh, mà thường gửi CV với cấu trúc tự do, kinh nghiệm đa dạng, kỹ năng viết theo nhiều alias và bằng cấp/chứng chỉ có cách diễn đạt không thống nhất. Nhà tuyển dụng không chỉ đăng một tiêu đề công việc, mà mô tả yêu cầu kỹ thuật, cấp bậc, địa điểm, hình thức làm việc, bằng cấp, chứng chỉ và nhiều tiêu chí mềm. Khi hai tập dữ liệu này cùng tồn tại dưới dạng văn bản dài, việc tìm kiếm bằng từ khóa thuần túy dễ bỏ sót ứng viên phù hợp và dễ đưa ra kết quả không giải thích được.

Các nền tảng ATS lớn như Greenhouse, LeverTRM và Workday Talent Acquisition được đặc tả trong tài liệu yêu cầu như các benchmark tham chiếu về candidate experience, pipeline management, automation, analytics và AI-assisted recruiting [1, 13, 14, 15]. Tuy nhiên, mục tiêu của JobConnect MVP không phải sao chép toàn bộ một ATS doanh nghiệp. MVP chọn một phần lõi đủ để chứng minh sản phẩm có giá trị: tài khoản candidate/recruiter/admin, CV/JD upload, structured profiles, public marketplace pool, two-way matching, application lifecycle, notifications và auditability.

Trong mô hình marketplace của JobConnect, một resume chỉ được nhà tuyển dụng thấy nếu resume có trạng thái **active**; một job chỉ được ứng viên thấy nếu job có trạng thái **published**. Quy tắc visibility này làm cho public pool rõ ràng và tránh nhầm lẫn giữa dữ liệu đang draft với dữ liệu đã sẵn sàng tham gia matching. Đây cũng là khác biệt quan trọng so với một hệ thống quản lý nội bộ theo tenant riêng: MVP không có private

company workspace, không có tenant isolation và không có quota/billing.

1.2 Nhóm người dùng và nhu cầu

JobConnect phục vụ ba nhóm người dùng. Candidate tạo tài khoản, cập nhật hồ sơ cá nhân, upload một hoặc nhiều CV, review dữ liệu đã parse, kích hoạt resume vào public pool, tìm kiếm hoặc match published jobs, apply và phản hồi recruiter invites. Recruiter tạo tài khoản dưới một employer organization, tạo hoặc upload JD, publish job, tìm kiếm/match active resumes, gửi lời mời và quản lý applications. Admin theo dõi users, content, parse jobs, matching activity, notifications và audit events; MVP admin là read-only trừ khi có yêu cầu moderation riêng [1].

Table 1.1: Vai trò người dùng và năng lực chính

Vai trò	Năng lực chính trong MVP
Candidate	Đăng ký, tạo profile, upload CV, review parsed resume, activate/archive resume, search/match jobs, apply, accept/reject invite, đọc notifications.
Recruiter	Đăng ký, tạo recruiter profile, liên kết organization, create/upload job, publish/close job, search/match resumes, send invites, quản lý applications.
Admin	Theo dõi users, documents, parse jobs, applications, invites, notifications và audit logs qua các endpoint read-only.

Từ góc nhìn candidate, hệ thống cần giúp họ biến một CV phi cấu trúc thành một resume entity có các field dùng được cho matching: `title`, `summary`, `experience`, `skills`, `location`, `job_type`, `seniority`, `education` và `certifications`. Candidate không bị buộc chỉ có một resume active; đặc tả cho phép nhiều active resumes, còn `is_primary` chỉ là `display/default-selection hint` [1].

Từ góc nhìn recruiter, hệ thống cần giúp chuyển một JD thành `job_posts` có `title`, `requirement`, `skills`, `location`, `job_type`, `seniority`, `education`, `required_certifications` và trạng thái `draft/published/closed`. Job chỉ vào public job pool khi `status = published`. Khi job đã `closed`, hệ thống không được nhận application hoặc invite mới.

1.3 Phát biểu bài toán

Bài toán của khóa luận có thể phát biểu như sau. Cho tập tài liệu CV do ứng viên upload và tập tài liệu JD do nhà tuyển dụng tạo hoặc upload, xây dựng một hệ thống backend có khả năng lưu trữ tài liệu gốc, trích xuất và chuẩn hóa thông tin, đưa resume/job hợp lệ vào public pool, tìm kiếm và so khớp hai chiều, giải thích kết quả ranking và hỗ trợ luồng ứng tuyển/lời mời.

Đầu vào của hệ thống gồm:

- Tài khoản người dùng với vai trò **candidate**, **recruiter**, **admin**.
- CV/JD file, tối thiểu hỗ trợ PDF; DOCX chỉ nên hỗ trợ khi extractor được kiểm thử.
- Dữ liệu manual create/update cho resume và job.
- Query keyword hoặc semantic search.
- Anchor **job_id** hoặc **resume_id** cho matching.

Đầu ra của hệ thống gồm:

- Structured resume/job records có enum canonical.
- Parse job status **queued**, **processing**, **succeeded**, **failed**.
- Embedding rows có **embedding_version**.
- Kết quả matching gồm rank, final score, component scores, exact skill overlap và reasoning.
- Application, invite, notification và audit log theo lifecycle nghiệp vụ.

Điểm khó nằm ở việc kết hợp semantic retrieval với rule nghiệp vụ. Một resume có thể semantic tương đồng với job nhưng vẫn không hợp lệ nếu job yêu cầu **senior** còn resume là **junior**, hoặc nếu JD required certifications không nằm trong CV certifications. Ngược lại, một pair pass hard filters vẫn cần score breakdown để phân biệt

mức độ phù hợp. Do đó JobConnect không dùng AI như một black box, mà tách thành eligibility, retrieval, scoring, reranking và reasoning.

1.4 Phạm vi MVP và các non-goals

Phạm vi MVP được xác định rõ trong `REQUIREMENTS.md`. Các thành phần in scope gồm authentication bằng email/password và JWT, roles, profile management, organization profile, CV/JD upload, parse pipeline, resume CRUD và activation, job CRUD và publishing, keyword search, semantic search, two-way matching, explainable ranking, application flow, recruiter invite flow, ATS-lite status lifecycle, notifications, audit logs và OpenAPI-documented production namespaces.

Table 1.2: MVP scope và non-goals

Trong phạm vi	Ngoài phạm vi
Email/password auth, JWT session, role checks	Billing, subscriptions, quota enforcement
Candidate/recruiter profile và organization profile	Tenant/company isolation hoặc private workspace
CV/JD upload, parse job, normalized fields, embeddings	Full ATS interview scheduling, offer approval, onboarding
Keyword search và semantic search tách riêng	CRM/nurture campaign
Job-to-resume và resume-to-job matching	Automated reject/advance solely by AI
Application, invite, notification, audit log	Production benchmark claims khi chưa có labeled evaluation data

Việc loại bỏ tenant isolation là một quyết định kiến trúc quan trọng. Trong MVP, all published jobs và all active candidate resumes nằm trong một shared public pool. Từ đó, logic authorization tập trung vào owner visibility cho draft/archived/closed records

và public visibility cho active/published records. Recruiter không cần private tenant để match active resumes, nhưng chỉ được quản lý job do mình sở hữu hoặc thuộc tổ chức được phép.

1.5 Các khái niệm nền tảng

1.5.1 ATS-lite workflow

ATS-lite trong JobConnect là tập workflow nhỏ nhưng đủ để minh họa vòng đời tuyển dụng. Candidate apply tạo một row trong `applications` với trạng thái `submitted`. Recruiter invite tạo một row `recruiter_invites` với trạng thái `pending`. Khi candidate accept invite, hệ thống tạo application nếu cặp (`job_id`, `resume_id`) chưa tồn tại. Recruiter có thể chuyển application từ `submitted` sang `shortlisted`, `rejected` hoặc `hired`; candidate có thể `withdrawn` nếu chưa có terminal recruiter decision. Mỗi status change phải tạo `application_events`.

1.5.2 Structured parsing

Structured parsing là bước chuyển đổi text phi cấu trúc thành field canonical. Parser phải nhận schema JSON và allowed enum labels, sau đó tạo output không invent unsupported enum values. Trong implementation target, parser có biên tích hợp dưới `backend/src/jobconnect/base.py` định nghĩa protocol, `local.py` là deterministic fallback, `openai.py` gọi endpoint OpenAI-compatible với `response_format=json_object` và validate enum trước khi trả dữ liệu [4].

1.5.3 Embedding và pgvector

Embedding biểu diễn title, skills, summary, experience hoặc requirement dưới dạng vector 384 chiều. Schema target lưu embedding trong hai bảng riêng: `candidate_resume_embeddings` và `job_post_embeddings`. Các vector field nullable để khi thiếu embedding, matching không fail toàn bộ; component tương ứng được chấm 0 và reasoning phải nêu rõ missing embedding [5, 9]. Provider boundary nằm dưới `backend/src/jobconnect/integrations`.

với local hash provider mặc định và OpenAI provider dùng `text-embedding-3-small` mặc định khi có credential.

1.5.4 Explainable matching

Explainable matching trong JobConnect không chỉ trả điểm tổng. Kết quả phải có hard-filter pass notes, exact skill overlap count, strongest score components, missing embedding notes và final score. Nếu LLM-generated reasoning được dùng, nó phải grounded in score breakdown và structured fields; LLM không được invent facts absent from CV/JD data [6]. Quy tắc này giúp kết quả matching có thể giải thích cho user và audit được khi xảy ra tranh chấp.

1.6 Yêu cầu trạng thái và enum canonical

Hệ thống validate enum ở API và persistence boundary. Các enum này không chỉ là giá trị hiển thị, mà quyết định visibility, lifecycle và matching eligibility.

Table 1.3: Enum canonical trong JobConnect MVP

Field	Giá trị hợp lệ
<code>users.role</code>	<code>candidate</code> , <code>recruiter</code> , <code>admin</code>
<code>users.status</code>	<code>active</code> , <code>invited</code> , <code>disabled</code>
<code>candidate_resumes.status</code>	<code>draft</code> , <code>active</code> , <code>archived</code>
<code>job_posts.status</code>	<code>draft</code> , <code>published</code> , <code>closed</code>
<code>applications.status</code>	<code>submitted</code> , <code>shortlisted</code> , <code>rejected</code> , <code>hired</code> , <code>withdrawn</code>
<code>recruiter_invites.status</code>	<code>pending</code> , <code>accepted</code> , <code>rejected</code>
<code>parse_jobs.status</code>	<code>queued</code> , <code>processing</code> , <code>succeeded</code> , <code>failed</code>
<code>notifications.status</code>	<code>unread</code> , <code>read</code>
<code>location</code>	<code>ha_noi</code> , <code>tp_hcm</code> , <code>da_nang</code>
<code>job_type</code>	<code>remote</code> , <code>fulltime</code> , <code>parttime</code>
<code>seniority</code>	<code>intern</code> , <code>fresher</code> , <code>junior</code> , <code>mid</code> , <code>senior</code> , <code>lead</code>
<code>education</code>	<code>lop_9</code> , <code>lop_12</code> , <code>dai_hoc</code> , <code>thac_si</code> , <code>tien_si</code>

Hai visibility rule là then chốt: `candidate_resumes.status = active` mới vào public candidate pool, và `job_posts.status = published` mới vào public job pool. Các trạng thái `draft`, `archived`, `closed` bị loại khỏi public search

và matching. Quy tắc này làm cho matching có tính dự đoán và tránh việc dữ liệu chưa review bị đưa vào kết quả.

1.7 Liên hệ với báo cáo cũ và định hướng viết lại

Knowledge Pack trong thư mục `report/knowledge` xác định rằng báo cáo mới giữ bố cục mới và tái sử dụng ý cũ ở mức luận điểm, không copy dài nguyên văn. Các heading cũ như “Thách thức của quy trình tuyển dụng hiện đại”, “Cơ hội từ kiến trúc RAG”, “Embedding models”, “Hybrid Retrieval & Scoring” và “Implementation vs Theory: Gaps Analysis” được chuyển thành seed nội dung cho các chương tương ứng. Trong bản báo cáo này, phần nội dung được viết lại theo source hiện tại của repository, nên không còn mô tả hệ thống như prototype RAG đơn lẻ mà chuyển thành JobConnect production MVP với API namespaces, schema và implementation matrix cụ thể.

CHAPTER 2

Mô hình và kỹ thuật nền tảng

2.1 Kiến trúc web API với FastAPI

Backend JobConnect dùng FastAPI làm lớp API runtime. Đây không phải lựa chọn được ghi chung chung là “web framework”, mà gắn trực tiếp với dependency `fastapi==0.115.6` trong `backend/requirements.txt`. FastAPI phù hợp với dự án vì cung cấp request validation, response model, dependency injection, OpenAPI generation và khả năng tổ chức router theo module. Trong repository, app entrypoint nằm tại `backend/src/jobconnect/app.py`, còn metadata và router registration nằm tại `backend/src/jobconnect/app.py`, còn production API router aggregator nằm tại `backend/src/jobconnect/modules/api/router.py`.

Theo tài liệu kiến trúc [3], runtime được chia thành các lớp: API layer, auth/authorization layer, domain services, data access layer, worker layer và external services. Cách chia này tránh để endpoint chứa toàn bộ logic nghiệp vụ. API layer chịu trách nhiệm validate request và trả schema rõ ràng; service layer xử lý trạng thái nghiệp vụ như activate resume, publish job, accept invite; data layer làm việc với PostgreSQL; worker layer xử lý parse/embedding; external services được bọc qua integration boundary.

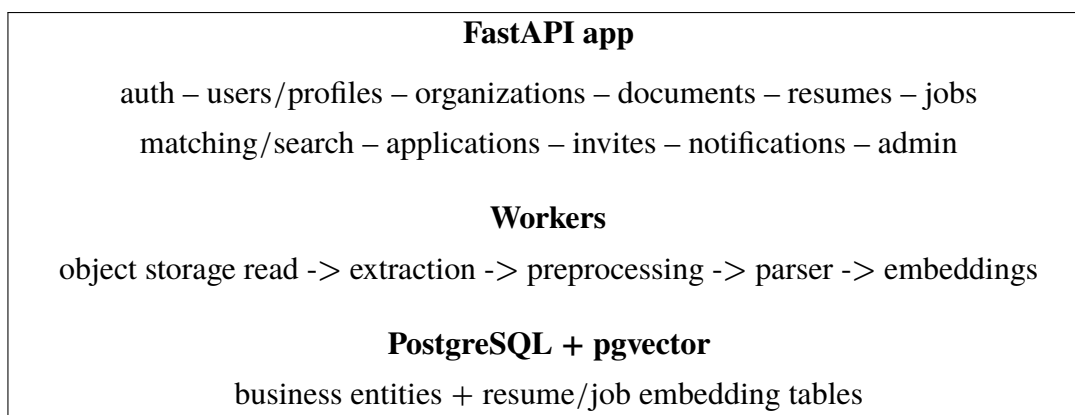


Figure 2.1: Kiến trúc runtime tổng quát của JobConnect

Nguồn: [3]..

FastAPI còn quan trọng vì OpenAPI là contract surface cho frontend. Tài liệu API LLD yêu cầu mọi route có request/response schema explicit, mọi enum phải thể hiện trong OpenAPI và mọi lỗi business phải đi qua error envelope thống nhất [10]. Vì vậy hệ thống không nên trả response tùy tiện theo từng endpoint; nó cần schema rõ cho `AuthResponse`, `CandidateResumeSummary`, `JobPostSummary`, `ApplicationSummary`, `InviteSummary`, semantic search responses và matching responses.

2.2 Pydantic, schema validation và error envelope

Pydantic 2.11.7 được dùng cho request/response model. Vai trò của Pydantic trong hệ thống không chỉ là kiểu dữ liệu Python, mà là lớp bảo vệ API contract. Các request như create resume, update job, semantic search, application status update hoặc invite request phải reject field lạ nếu endpoint không cho phép metadata JSON. Theo implementation matrix, Pydantic models dùng `extra="forbid"` ở nhiều nơi, và PATCH endpoints đã được điều chỉnh theo hướng partial update bằng `exclude_unset` [11].

Error envelope target có dạng:

Listing 2.1: Error envelope target trong API contract

```

{
  "error": {
    "code": "validation_error",
    "message": "Request validation failed.",
    "fields": {

```

```
    "email": "Invalid email address."
  },
  "request_id": "req_01hxyz"
}
```

Cách chuẩn hóa lỗi giúp frontend xử lý thống nhất các trường hợp 400, 401, 403, 404, 409, 413, 415, 422 và 500. Với workflow upload, 413 dùng cho file vượt giới hạn, 415 cho MIME type không hỗ trợ. Với lifecycle, 409 phù hợp cho duplicate invite, duplicate application hoặc invalid transition.

2.3 PostgreSQL và pgvector

PostgreSQL là source of truth cho business entities: users, organizations, candidate_profiles, recruiter_profiles, candidate_resumes, job_posts, uploaded_documents, parse_jobs, applications, application_events, recruiter_invites, notifications và audit_logs. pgvector được dùng trong cùng PostgreSQL để lưu semantic vectors cho resume/job fields. Điều này giúp hệ thống giữ transactional business data và semantic retrieval data trong một data store quản lý được ở MVP.

Schema LLD quy định primary key dùng BIGINT GENERATED BY DEFAULT AS IDENTITY; timestamps dùng TIMESTAMPTZ; enum-like values dùng TEXT hoặc VARCHAR với CHECK constraints thay vì PostgreSQL native enum; normalized skills/certifications dùng TEXT[] với default {}; vector columns dùng VECTOR(384) và nullable [9]. Việc để vector nullable là thiết kế có chủ đích: thiếu embedding không làm fail toàn bộ matching run, mà component tương ứng được score 0.

Table 2.1: Nhóm bảng chính trong schema JobConnect

Nhóm	Bảng cụ thể
Identity và profile	users, candidate_profiles, recruiter_profiles, organizations
Marketplace entities	candidate_resumes, job_posts
Document pipeline	uploaded_documents, parse_jobs
Semantic vectors	candidate_resume_embeddings, job_post_embeddings
ATS-lite	applications, application_events, recruiter_invites
Operations	notifications, audit_logs

2.4 Pipeline xử lý tài liệu CV/JD

Pipeline ingestion là một trong những phần quan trọng nhất của JobConnect. Theo HLD ingestion, luồng core là: upload file, store original in object storage, create `uploaded_documents`, create `parse_jobs`, worker marks processing, extract raw text, preprocess text, normalize skills, LLM structured parse, update resume/job fields, expose review payload, save user corrections, generate embeddings, mark succeeded or failed [4].

```

upload file -> object storage -> uploaded_documents -> parse_jobs
-> extract text -> preprocess -> skill normalization -> LLM/local parser
-> draft resume/job fields -> user review -> embeddings -> active/published pool

```

Figure 2.2: Pipeline xử lý CV/JD từ file đến public pool

Nguồn: [4].

File handling có một invariant quan trọng: parse failure không được xóa original file. Database chỉ lưu metadata và object key/file URL, còn object storage là source of truth cho file gốc. Upload endpoint trả về sau khi file persistence và parse job creation hoàn tất; parse chạy async. Điều này giúp API upload có latency kiểm soát được và tránh giữ kết nối quá lâu trong khi extraction/LLM/embedding chạy.

2.4.1 Preprocessing

Preprocessing phải chạy sau raw text extraction và trước structured parsing. Minimum behavior gồm remove null bytes, invalid control characters, normalize Unicode to NFC cho tiếng Việt, collapse excessive whitespace, preserve technical terms/certification names và không dịch toàn bộ document. Quy tắc “không dịch toàn bộ document” rất quan trọng: nếu dịch trước khi parse, thuật ngữ như “Spring” có thể bị hiểu sai thành mùa xuân, hoặc certification name bị đổi nghĩa.

2.4.2 Skill normalization

Skill normalization xử lý alias trước hard-filter extraction và embedding. Ví dụ `reactjs`, `react.js`, `React` được normalize thành `react`; `nodejs`, `node.js` thành `nodejs`; `pytorch`, `PyTorch` thành `pytorch`. Tài liệu yêu cầu gợi ý dictionary MVP khoảng 50-100 high-impact entries [1]. Trong code layout hiện tại, phần này tương ứng với `backend/src/jobconnect/modules/documents/skill_normalize`

2.4.3 LLM structured parsing

Parser boundary nằm tại `backend/src/jobconnect/integrations/llm/.base.py` định nghĩa `LLMParser` protocol với `parse_resume`, `parse_job` và `parser_version`. `local.py` là deterministic keyword parser với `parser_version = "local-v1"`. `openai.py` gọi OpenAI-compatible chat completions endpoint với `response_format=json_object`, sau đó validate enum field trước khi trả về. Environment variables gồm `LLM_PROVIDER`, `OPENAI_API_KEY`, `OPENAI_MODEL`, `OPENAI_BASE_URL`, `OPENAI_TIMEOUT_SECONDS`. Default model trong docs là `gpt-4o-mini` [4].

Thiết kế provider boundary làm cho hệ thống có thể chạy local không cần credential nhưng vẫn sẵn sàng chuyển sang provider thật. Nếu `LLM_PROVIDER` không hợp lệ hoặc thiếu `OPENAI_API_KEY`, factory fallback về local parser. Nếu parser provider ném `ParserError`, worker đánh dấu parse job failed với `error_code = llm_parse_failed` và bảo toàn file gốc.

2.5 Embedding provider và semantic search

Embedding provider boundary nằm tại `backend/src/jobconnect/integrations/`
`base.py` định nghĩa `EmbeddingProvider` protocol, `EmbeddingError` và constant `EMBEDDING_DIM = 384`. `local.py` bọc `LocalHashEmbeddingProvider` dựa trên SHA-256-seeded bag-of-words embedder với `embedding_version = "hash-v1"`. `openai.py` gọi endpoint `/v1/embeddings`, dùng `dimensions=384` và default model `text-embedding-3-small`; vector sai chiều phải raise `EmbeddingError`, không được silently padding [6].

Table 2.2: Cấu hình embedding provider

Biến môi trường	Default	Ý nghĩa
<code>EMBEDDING_PROVIDER</code>	<code>local</code>	Chọn <code>local</code> hoặc <code>openai</code> ; unknown fallback về local.
<code>OPENAI_EMBEDDING_API_KEY</code>	<code>unset</code>	Credential riêng cho embedding; fallback sang <code>OPENAI_API_KEY</code> .
<code>OPENAI_EMBEDDING_MODEL</code>	<code>text-embedding-3-small</code>	Model name đưa vào <code>embedding_version</code> .
<code>OPENAI_EMBEDDING_BASE_URL</code>	OpenAI API	Cho endpoint compatible hoặc proxy.
<code>OPENAI_EMBEDDING_TIMEOUT_SECONDS</code>	30	Timeout HTTP request.

Semantic search được tách khỏi matching. Job semantic search dùng query theo ý định ứng viên hoặc mô tả CV-style và rank theo JD requirement embeddings. Resume semantic search rank theo CV summary/experience embeddings, không rank title embeddings. Score semantic search là retrieval relevance, không phải final matching score [1, 6]. Sự tách biệt này ngăn người dùng hiểu nhầm rằng mọi kết quả semantic search đều là match đủ điều kiện nghiệp vụ.

2.6 Matching pipeline

Matching pipeline nhận một anchor đã đủ visibility: published job cho job-to-resume, active resume cho resume-to-job. Sau khi validate auth, role, ownership và anchor visibility, hệ thống lấy opposite pool, apply hard filters, score và optional rerank.

Matching result không tạo application hoặc invite; nó chỉ là recommendation.

2.6.1 Hard filters

Hard filters đảm bảo cặp JD-CV có thể chấp nhận về mặt nghiệp vụ trước khi tính score semantic. Các rule gồm:

Table 2.3: Hard filters trong matching

Filter	Rule
job_type	Job type phải match. Nếu JD là <code>remote</code> , location filter được bỏ qua.
location	Với job không remote, CV và JD phải có cùng location.
seniority	Seniority phải match chính xác.
education	CV education phải lớn hơn hoặc bằng job required education theo thứ tự <code>lop_9 < lop_12 < dai_hoc < thac_si < tien_si</code> .
certifications	JD required certifications phải là subset của CV certifications.
Missing data	Thiếu dữ liệu hard-filter trên một phía làm pair fail hard filter.

2.6.2 Scoring deterministic

Score formula giữ lại công thức prototype nhưng được đặt vào bối cảnh production MVP:

$$\begin{aligned}
 final_score = & 0.35 \times title_sim + 0.35 \times skills_sim \\
 & + 0.20 \times req_exp_sim + 0.10 \times req_summary_sim \\
 & + bonus_exact_skill - penalty_missing_required
 \end{aligned} \tag{2.1}$$

$$skills_sim = 0.6 \times semantic_skills + 0.4 \times exact_overlap_ratio \quad (2.2)$$

Default `top_k = 10`, allowed range `1..50`; default `min_score = 0.7`, allowed range `0..1`. Bonus và penalty hiện giữ 0 đến khi có measured rule [1, 6]. Tie-break deterministic là `final_score DESC`, sau đó `resume_id ASC` cho job anchors hoặc `job_id ASC` cho resume anchors. Quy tắc tie-break giúp cùng dữ liệu và cùng model version luôn trả thứ tự ổn định.

2.6.3 Reranking và reasoning

Runtime hiện tại cố gắng local cross-encoder rerank trên top deterministic candidates, dùng `sentence-transformers==3.0.1`. Nếu reranker lỗi hoặc unavailable, matching fallback về deterministic scoring và đưa warning trong runtime metadata [11]. Đây là cách hợp lý cho MVP vì không làm matching fail chỉ vì model phụ không sẵn sàng.

Reasoning phải grounded in score breakdown và structured fields. Một reasoning tốt nên nói rõ kỹ năng nào overlap, hard filters nào pass, component nào mạnh/yếu và có thiếu embedding nào không. LLM-generated reasoning được phép, nhưng chỉ khi không invent fact. Điều này đặc biệt quan trọng trong tuyển dụng, vì một gợi ý sai hoặc không giải thích được có thể ảnh hưởng đến quyết định của người dùng.

2.7 Authentication, authorization và auditability

Authentication dùng email/password và JWT session. Implementation matrix ghi nhận custom HMAC JWT với `sub`, `role`, `iat`, `exp`; token hết hạn bị reject với `expired_token` token thiếu `exp` cũng bị reject [11]. Disabled users không được publish, activate, apply, invite hoặc run matching.

Authorization theo role và ownership. Candidates quản lý profile, documents, resumes, applications, invites và notifications của mình. Recruiters quản lý jobs và recruiting actions liên quan đến job họ sở hữu hoặc được phép quản lý theo organization. Admin

đọc dữ liệu monitoring, nhưng MVP không thêm destructive moderation. Privacy rule yêu cầu recruiters chỉ search/match active resumes, candidates chỉ search/match published jobs, và contact data của candidate phải hạn chế theo role/flow cần thiết [8].

Audit logs là một phần quan trọng để hệ thống có thể vận hành. Các event cần audit gồm resume activated/archived, job published/closed, parse job failed, candidate applied, recruiter invite sent, invite accepted/rejected, application status changed và admin monitoring access where useful. Audit event cần actor user ID nếu có, target entity, event type, timestamp và metadata JSON.

CHAPTER 3

Phân tích và thiết kế hệ thống đề xuất

3.1 Tổng quan kiến trúc JobConnect MVP

JobConnect được thiết kế như một backend production MVP có module boundaries rõ ràng. Repository không còn tập trung vào prototype route `/api/v2/prototype/*`; runtime hiện exposing production namespaces dưới `/api/*`. Theo README và HLD overview, legacy prototype code đã được loại khỏi runtime, còn legacy docs chỉ dùng như tài liệu tham chiếu migration [12, 2].

Backend layout chính nằm dưới `backend/src/jobconnect`. Cấu trúc này có bốn vùng: `core` cho config, database, security, exceptions, logging; `common` cho dependencies, pagination, responses, utils, constants; `integrations` cho storage, LLM, embedding, rerank, pgvector; và `modules` cho từng feature domain. Cách tổ chức này cho phép mỗi nghiệp vụ có router, schemas và service riêng mà không lẫn với hạ tầng.

Table 3.1: Mapping thư mục backend sang trách nhiệm kiến trúc

Đường dẫn	Trách nhiệm
<code>backend/src/jobconnect/main.py</code>	Main app entrypoint.
<code>backend/src/jobconnect/app.py</code>	App metadata và router registration.
<code>core/</code>	Config, database connection, security, exceptions, logging.
<code>common/</code>	Shared dependencies, pagination, response helpers, constants, utils.
<code>integrations/storage/</code>	Local/cloud object storage boundary.
<code>integrations/llm/</code>	Local/OpenAI-compatible parser boundary.
<code>integrations/embedding/</code>	Local hash/OpenAI embedding provider boundary.
<code>integrations/rerank/</code>	Local cross-encoder reranker boundary.

Đường dẫn	Trách nhiệm
<code>modules/*/</code>	Domain routers, schemas và services cho từng feature.

3.2 Thiết kế module nghiệp vụ

Feature modules trong `backend/src/jobconnect/modules` phản ánh trực tiếp sản phẩm. Module `auth` xử lý register/login/logout, password hashing và JWT issuance. Module `users` phục vụ current user bootstrap. Module `organizations` quản lý employer organization, bao gồm predefined Independent organization dùng cho lựa chọn “Khác”. Module `resumes` và `jobs` là hai matching entities chính. Module `documents` xử lý upload, parse jobs, extraction/preprocessing và worker. Module `matching` chứa hard filters, scoring, reasoning, embedding helper và vector search. Module `applications`, `invites`, `notifications` và `admin` triển khai workflow ATS-lite và monitoring.

Table 3.2: Feature modules và API namespace liên quan

Module	Namespace	Vai trò
auth	/api/auth/*	Đăng ký, đăng nhập, logout, phát JWT.
users	/api/me	Bootstrap thông tin user và role-specific profile.
organizations	/api/organizations/*	Employer profile và Independent organization lookup.
resumes	/api/candidate/resumes/*	Resume CRUD, activate/archive, search, semantic search.
jobs	/api/jobs/*	Job CRUD, publish/close, search, semantic search.
documents	/api/documents/*	Upload file, download URL, parse job status/retry.
matching	/api/matching/*	Run job->resume và resume->job matching.
applications	/api/applications/*	Apply và status transition.
invites	/api/invites/*	Recruiter invite, candidate accept/reject.
admin	/api/admin/*	Read-only monitoring.

Thiết kế module này giúp báo cáo và code có cùng ngôn ngữ. Khi nói về “candidate resume lifecycle”, ta có thể chỉ thẳng vào module **resumes**; khi nói về “parse worker”, ta chỉ vào module **documents**; khi nói về “ranking explainability”, ta chỉ vào module **matching**. Điều này giảm rủi ro viết báo cáo ở mức trừu tượng xa code.

3.3 Thiết kế dữ liệu

Thiết kế dữ liệu JobConnect dựa trên ba nhóm invariant. Nhóm đầu tiên là invariant định danh và profile: một user có role, status và profile tương ứng. Nhóm thứ hai là invariant public pool: resume active và job published mới được search/match. Nhóm thứ ba là invariant lifecycle: apply/invite/status change phải tạo event, notification và audit tùy flow.

3.3.1 Identity và organization

`users` lưu email unique, password hash, role, status và timestamps. Password không bao giờ lưu plain text. `candidate_profiles` chứa full name, phone, current location, total experience years và headline; `total_experience_years` chỉ là display metadata, không phải hard filter hoặc score input trong MVP [1]. `recruiter_profiles` liên kết recruiter user với organization. `organizations` có name, slug, logo URL và about; migration/seed phải tạo một row Independent có slug `independent` cho lựa chọn “Khác”.

3.3.2 Resume và job

`candidate_resumes` là matching entity phía CV. Các field required gồm title, summary, experience, skills, location, job_type, seniority, education, certifications, is_primary, status. `job_posts` là matching entity phía JD, gồm title, requirement, skills, location, job_type, seniority, education, required_certifications, status, published_at, expires_at. Hai bảng này có GIN indexes cho skills/certifications và partial indexes cho public pool [9].

3.3.3 Documents, parse jobs và embeddings

`uploaded_documents` lưu metadata file gốc: owner, document type, object key/file URL, original filename, MIME type, size và link đến resume/job nếu có. `parse_jobs` lưu status, extracted text hoặc object key, parser version, embedding version requested, error code/message và timestamps. Parse job có thể chưa có `resume_id` hoặc `job_id` khi queued/processing; succeeded parse job phải link đúng một target entity [9].

Embedding tables tách riêng khỏi business entity để hỗ trợ versioning và backfill. `candidate_resume_embeddings` có `emb_title`, `emb_skills`, `emb_summary`, `emb_experience`; `job_post_embeddings` có `emb_title`, `emb_skills`, `emb_requirement`. Mỗi row có `embedding_version`. Khi provider/model thay đổi, operator có thể tìm rows cần re-embedding bằng điều kiện `WHERE embedding_version <> '<new>'`.

3.3.4 Applications, invites, notifications và audit

`applications` có unique constraint trên (`job_id`, `resume_id`). Điều này đảm bảo một resume không apply trùng cùng một job. `application_events` append-only status history. `recruiter_invites` lưu lời mời từ recruiter cho một resume vào một job, với status `pending/accepted/rejected`. `notifications` lưu thông báo in-app và email delivery status. `audit_logs` lưu business events quan trọng.

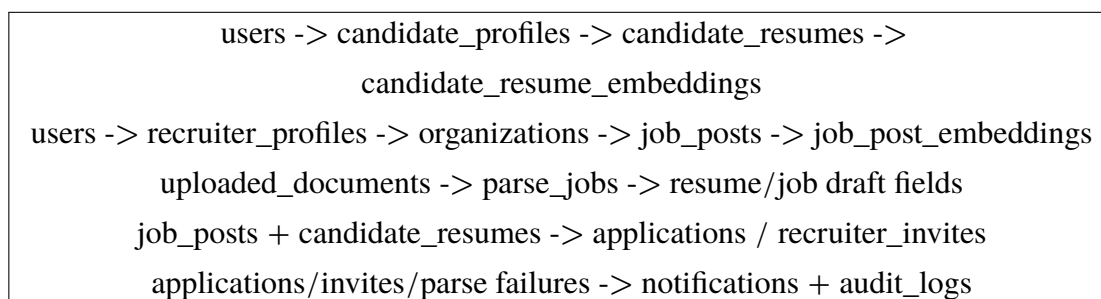


Figure 3.1: Quan hệ logic giữa các nhóm entity

Nguồn: [5, 9].

3.4 Thiết kế API

Base path production là `/api`. Đây là breaking replacement cho prototype-only routes dưới `/api/v2/prototype/*`. API contract quy định ID là JSON number đại diện cho BIGINT, timestamp là ISO 8601 có timezone, request/response schema explicit và unknown request fields bị reject trừ endpoint cho phép metadata JSON [10].

Table 3.3: Production API namespaces

Namespace	Mục đích
<code>/api/auth/*</code>	Register, login, logout, JWT session.
<code>/api/me</code>	Current account và role-specific bootstrap data.
<code>/api/candidate/profile</code>	Candidate profile read/upsert.
<code>/api/candidate/resumes/*</code>	Resume CRUD, activation/archive, search, semantic search.
<code>/api/recruiter/profile</code>	Recruiter profile read/upsert.
<code>/api/organizations/*</code>	Organization create/update/list/detail.

Namespace	Mục đích
/api/jobs/*	Job CRUD, publish/close, search, semantic search.
/api/documents/*	Multipart upload, document detail, download URL, parse jobs.
/api/matching/*	Job anchor và resume anchor matching.
/api/applications/*	Candidate apply, list/detail, status transition.
/api/invites/*	Recruiter invite, candidate accept/reject.
/api/notifications/*	Notification list/read/read-all.
/api/admin/*	Admin monitoring users/documents/parse jobs/applications/invites/notifications/audit logs.

Một điểm thiết kế đáng chú ý là response list/detail cho frontend cần denormalized display summaries ở các nơi phù hợp. Ví dụ job list/search responses target cần `organization_name`, `organization_logo_url`, `organization_slug` để job cards không phải fetch organization từng row. Application/Invite summaries target cần `job_summary` và `resume_summary` để table/detail ổn định [7, 10].

3.5 Thiết kế luồng runtime

3.5.1 Candidate CV flow

Luồng candidate bắt đầu từ register/login, sau đó candidate tạo profile, upload CV hoặc tạo resume thủ công. Upload CV tạo document và parse job. Khi parse succeeded, candidate review parsed fields và save corrections vào draft resume. Resume chỉ vào public pool khi candidate gọi activate. Sau đó candidate có thể search Job Market hoặc run matching từ một active resume để tìm published jobs. Apply tạo application với status **submitted**; responding invite có thể accept hoặc reject.

register/login -> profile -> upload CV -> parse job -> review -> activate resume
-> search/match published jobs -> apply hoặc respond invite

Figure 3.2: Luồng candidate từ CV đến ứng tuyển

Nguồn: [7]..

3.5.2 Recruiter JD flow

Recruiter đăng ký, chọn organization, tạo recruiter profile, tạo hoặc upload JD. JD upload cũng tạo document và parse job. Sau khi review parsed job fields, recruiter publish job để đưa vào public job pool. Recruiter chạy Talent Market search hoặc matching từ published job để tìm active resumes. Nếu phù hợp, recruiter tạo invite. Invite pending không tạo application; accept mới tạo application nếu chưa tồn tại.

3.5.3 Matching flow

Matching flow bắt đầu bằng auth và role validation. Với job anchor, recruiter phải sở hữu published job hoặc admin bypass; với resume anchor, candidate phải sở hữu active resume hoặc admin bypass. Hệ thống lấy opposite pool theo visibility, apply hard filters, score, optional rerank, rồi trả ranked results với reasoning. Matching không có side effect tạo application/invite.

3.5.4 Application và invite flow

Application flow có side effects rõ: candidate apply hoặc accepted invite tạo application, append application event, create notification và write audit event. Status update cũng append event, notify candidate hoặc recruiter theo flow và write audit. Email failure không rollback business transaction, chỉ log để retry hoặc operational review [8].

3.6 Thiết kế bảo mật và quyền truy cập

Thiết kế bảo mật của JobConnect đặt trọng tâm vào role, ownership và visibility. Candidate chỉ quản lý tài nguyên của mình; recruiter chỉ quản lý job và recruiting actions được phép; admin read-only monitoring. Disabled users bị chặn khỏi các active marketplace actions. Hệ thống không có tenant isolation trong MVP, nên không được thêm logic private workspace ẩn làm sai product model.

Privacy của candidate là điểm cần lưu ý. Recruiters có thể search/match active

resumes, nhưng candidate contact fields không nên hiển thị rộng rãi nếu chưa có application hoặc invite flow cần thiết. Điều này cần được thể hiện ở response DTO, không chỉ ở frontend. Nếu runtime dùng cùng `ResumeDetail` cho candidate và recruiter mà chưa tách privacy DTO, đây là điểm cần hardening trong các phiên bản sau.

3.7 Thiết kế observability

Minimum operational signals gồm parse queue state, parse failure reasons, matching request count/latency, notification creation, email delivery failures, application/invite lifecycle events và API error rates cho auth/upload/parse/matching/application flows [8]. Trong MVP, audit log và notification table đóng vai trò nền tảng để quan sát business events. Runtime metrics trong matching gồm `total_ms`, `retrieval_ms`, `filter_ms`, `scoring_ms`, `rerank_ms`, candidate counts before/after hard filters, rerank-applied flag và warnings [6].

Thiết kế observability này có ý nghĩa thực tế. Nếu matching trả ít kết quả, hệ thống cần biết do candidate pool nhỏ, hard filters quá chặt, embedding thiếu hay min score quá cao. Nếu parse job failed nhiều, operator cần phân biệt lỗi extraction, LLM parse hay embedding. Nếu email không gửi được, business transaction vẫn thành công nhưng operational review cần thấy `email_delivery_status`.

3.8 Điểm sáng tạo và khác biệt của mô hình đề xuất

Điểm sáng tạo của JobConnect không nằm ở việc dùng một model AI duy nhất, mà nằm ở cách kết hợp rule nghiệp vụ, structured data và semantic representation. Hệ thống không giao toàn bộ quyết định cho LLM. LLM/parser chỉ chuyển text thành field canonical; hard filters quyết định eligibility; vector retrieval tạo candidate set; deterministic formula tính score; reranker chỉ là lớp tăng chất lượng tùy chọn; reasoning phải grounded. Thiết kế này giúp hệ thống vừa tận dụng ngữ nghĩa của CV/JD, vừa giữ được tính kiểm soát và khả năng giải thích.

Khác biệt thứ hai là mô hình two-way marketplace. Nhiều hệ thống chỉ tối ưu

job-to-candidate matching cho recruiter. JobConnect yêu cầu cả candidate anchor và recruiter anchor: candidate dùng active resume để tìm published jobs, recruiter dùng published job để tìm active resumes. Điều này ảnh hưởng trực tiếp đến API, authorization, visibility và tie-break: resume anchor sort tie theo `job_id ASC`, job anchor sort tie theo `resume_id ASC`.

Khác biệt thứ ba là thiết kế rõ future provider boundaries. Local deterministic parser, local hash embedding và local reranker giúp dev/test chạy không cần credential. OpenAI-compatible parser và embedding provider sẵn sàng dùng khi có credential. Storage adapter mặc định local filesystem, nhưng cloud adapter có thể slot in qua protocol. Cách này giảm rủi ro vendor lock-in và giúp MVP vẫn chạy được trong môi trường phát triển.

CHAPTER 4

Cài đặt thực nghiệm và đánh giá

4.1 Môi trường và công nghệ triển khai

Repository JobConnect hiện tập trung vào backend FastAPI. Theo README, backend API chạy tại `http://localhost:8000`, PostgreSQL ở host port 5433, schema migration nằm tại `backend/db/migrations/001_production_mvp.sql`, và verification backend dùng `python -m unittest discover -s tests` trong Docker Compose cho runtime backend [12]. Riêng workflow LaTeX của báo cáo này chạy local bằng `xelatex` theo rule riêng trong `AGENTS.md`.

Các dependency runtime cụ thể được lấy từ `backend/requirements.txt`:

Table 4.1: Dependency backend cụ thể trong dự án

Package	Version	Vai trò
fastapi	0.115.6	API runtime, routing, OpenAPI generation.
uvicorn[standard]	0.24.0	ASGI server cho FastAPI.
pydantic	2.11.7	Request/response schema validation.
psycopg[binary]	3.2.3	PostgreSQL driver.
numpy	1.26.4	Vector/math utilities cho embedding/scoring.
python-multipart	0.0.20	Multipart upload cho <code>/api/documents</code> .
pypdf	4.3.1	PDF text extraction cho parse worker.
httpx	0.28.1	TestClient support và HTTP integration.

Package	Version	Vai trò
sentence-transformers	3.0.1	Local multilingual cross-encoder reranker.

Bảng trên cho thấy phần cài đặt không dừng ở mức “ngôn ngữ Python”. Mỗi dependency đều gắn với một boundary cụ thể: FastAPI/Pydantic cho API contract, psycopg cho PostgreSQL, python-multipart cho upload, pypdf cho extraction, sentence-transformers cho rerank. Điều này quan trọng vì báo cáo kỹ thuật cần phản ánh đúng sản phẩm đang có, không dùng mô tả chung có thể áp dụng cho bất kỳ dự án nào.

4.2 Cài đặt backend runtime

App runtime bắt đầu từ `backend/src/jobconnect/main.py` và `backend/src/jobconnect/app.py`. App mount production API routers dưới base prefix `/api`. Health route `/api/health` và root `/` là runtime-only surfaces. Domain router aggregator nằm ở `backend/src/jobconnect/domain_router.py` file này import các router domain, xây dựng `ALL_API_ROUTERS` và re-export compatibility symbols phục vụ tests/entrypoints.

Listing 4.1: Backend layout rút gọn

```
backend/src/jobconnect/
  app.py
  main.py
  core/
  common/
  integrations/
    embedding/
    llm/
    pgvector/
    rerank/
    storage/
  modules/
    auth/
    users/
    organizations/
    jobs/
```

```
resumes/  
documents/  
matching/  
applications/  
invites/  
notifications/  
admin/  
system/
```

Từng module domain thường có `router.py`, `schemas.py` và `service.py`. Ví dụ module `matching` có thêm `filters.py`, `scoring.py`, `reasoning.py`, `vector_search.py`, `embedding.py` và `models.py`. Module `documents` có `extractor.py`, `preprocessor.py`, `skill_normalizer.py`, `local_parser.py` và `worker.py`. Cấu trúc này phản ánh trực tiếp pipeline nghiệp vụ.

4.3 Cài đặt cơ sở dữ liệu

Migration production MVP nằm tại `backend/db/migrations/001_production_mv`. Theo README, data không auto-load chỉ bằng `docker compose up`; cần chạy `docker compose exec backend python db/apply_migrations.py`. Volume `postgres_data` giữ dữ liệu qua `down/up`; chỉ `down -v` mới xóa volume. Điều này ảnh hưởng đến việc thực nghiệm: nếu test thủ công trên database đã có dữ liệu, cần biết dữ liệu có thể tồn tại từ lần chạy trước.

Schema production dùng PostgreSQL + pgvector. Các bảng embedding dùng `VECTOR(384)`. Mỗi bảng quan trọng có index tương ứng: `users` theo `role/status`, `resumes` theo `owner/status` và `public pool`, `jobs` theo `organization/recruiter/status/published`, GIN index cho `skills/certification`, `parse jobs` theo `status/created`, `applications` theo `job/resume/candidate/status` và `audit logs` theo `actor/target/event type` [9].

Table 4.2: Một số invariant database quan trọng

Invariant	Ý nghĩa thực nghiệm
(<code>job_id</code> , <code>resume_id</code>) unique trong <code>applications</code>	Không cho duplicate application cho cùng resume và job.
<code>status = active</code> cho resume public	Recruiter search/match chỉ thấy active resumes.
<code>status = published</code> cho job public	Candidate search/match chỉ thấy published jobs.
Embedding version bắt buộc	Cho phép backfill/re-embedding khi đổi provider/model.
Vector nullable	Thiếu embedding không làm matching fail toàn bộ.
Audit append event	Các lifecycle actions có dấu vết vận hành.

4.4 Cài đặt document upload và parse worker

Theo implementation matrix, `POST /api/documents` đã chuyển sang multi-part upload, stream file qua Storage adapter, insert `uploaded_documents`, insert queued `parse_jobs` và write audit event. Default storage backend là local filesystem dưới `STORAGE_LOCAL_ROOT`; future cloud adapter chỉ cần implement cùng protocol [11]. Endpoint reject unsupported MIME với 415 và oversize file với 413.

Document detail endpoint đã include linked parse jobs, và download URL endpoint trả qua Storage adapter. Local adapter trả `local://<object_key>`; future cloud adapter sẽ trả presigned HTTPS URL. Retry parse job tạo queued parse job mới và audit `parse_job_retried`.

Tuy nhiên implementation matrix cũng ghi rõ parse job response hiện vẫn thin so với target. Target review payload cần normalized fields, hard-filter fields, extracted text/reference, parser metadata, embedding metadata, warnings và linked `resume_id/job_id`. Đây là khoảng cách quan trọng với frontend parse review page. Vì vậy báo cáo đánh giá phần document pipeline là “foundation aligned, review payload partial” chứ không tuyên

bổ đã hoàn chỉnh toàn bộ ingestion target.

4.5 Cài đặt matching và semantic search

Runtime matching hiện có hai endpoint chính: `POST /api/matching/jobs/{job_id}` cho recruiter/admin và `POST /api/matching/resumes/{resume_id}/run` cho candidate/admin. Cả hai endpoint đọc anchor, validate ownership và visibility, lấy opposite pool, apply hard filters, score deterministic, attempt local rerank và trả warning metadata nếu rerank fallback [11].

Matching request có `top_k` default 10, `min_score` default 0.7 và `rerank` tương thích. Runtime Slice 8 ghi nhận rerank được attempted regardless request flag và fallback gracefully trên provider failure. Điều này có nghĩa API không phụ thuộc tuyệt đối vào reranker; nếu sentence-transformers model không load được hoặc lỗi, kết quả deterministic vẫn trả về.

Table 4.3: Các bước matching trong runtime

Bước	Cài đặt/ý nghĩa
Anchor validation	Published job cho job anchor; active resume cho resume anchor; ownership theo role.
Opposite pool	Active resumes hoặc published jobs tùy chiều matching.
Hard filters	Job type/location/seniority/education/certifications; missing data fail pair.
Scoring	Công thức deterministic với title, skills, req-exp và req-summary components.
Rerank	Local cross-encoder trên top window; fallback warning nếu unavailable.
Reasoning	Tạo giải thích dựa trên score breakdown và structured fields.

Semantic search cũng đã có endpoints `/api/candidate/resumes/semantic-search` và `/api/jobs/semantic-search`. Resume semantic search compute query embedding, rank theo CV summary/experience embeddings; job semantic search rank theo JD requirement embedding. Provider embedding failures trả `503 embedding_unavailable` envelope ở các route semantic search đã hardened trong Slice 8 [11]. Đây là một điểm

cài đặt cụ thể cho thấy hệ thống đã xử lý failure provider ở bề mặt API, không chỉ ghi trong thiết kế.

4.6 Cài đặt auth, authorization và lifecycle

Auth runtime có `POST /api/auth/register`, `POST /api/auth/login`, `POST /api/auth/logout` và `GET /api/me`. Implementation matrix ghi nhận register/login tạo hoặc đọc user, hash password, issue JWT với `expires_in`; logout stateless trả `204`; `/api/me` trả user và role-specific bootstrap profile/organization [11].

Authorization Slice 2 đã xử lý các điểm quan trọng: admin không đọc candidate/recruiter profile qua endpoint own-profile nữa; organization update kiểm tra recruiter membership; job matching yêu cầu recruiter ownership; resume matching yêu cầu candidate ownership; disabled users bị reject bởi `require_active`. Đây là các điểm cần nhấn mạnh vì nếu thiếu ownership check, matching có thể leak dữ liệu hoặc cho phép user thao tác sai tài nguyên.

Lifecycle Slice 3 và Slice 9 đã bổ sung partial PATCH, transition guard cho job publish/close, resume activate/archive, application status graph và terminal-state lock. Invite creation reject duplicate pending invite với `409 duplicate_invite`; invite accept trả explicit response `{ invite, application }`; closed-job blocks còn được tracked như follow-up trong Slice 9 matrix.

4.7 Cài đặt notifications và audit

Notifications runtime có list, read và read-all. `POST /api/notifications/read-all` trả `updated_count`; đây là breaking fix so với key cũ `updated` trong matrix. Application apply/status update và invite send/accept/reject tạo notification rows. Tuy nhiên email provider chưa được gọi; notification rows dùng `email_delivery_status = queued`. Target yêu cầu email delivery failure không rollback business transaction, nên phần email adapter là hướng hoàn thiện tiếp theo [8, 11].

Audit runtime được dùng cho nhiều lifecycle actions: organization create/update,

resume activate/archive, job publish/close, document upload/retry, application apply/status, invite sent/accepted/rejected và một số admin monitoring access. Matrix ghi nhận audit consistency vẫn là medium/low priority gap: một số admin reads audit access, một số chưa. Điều này không làm hỏng core workflow, nhưng ảnh hưởng đến observability và governance khi hệ thống chuyển production.

4.8 Bộ kiểm thử hiện có

Thư mục `backend/tests` cung cấp test coverage cho các phần then chốt: app surface, auth, authorization, API contract, documents, embedding provider, LLM parser, matching helpers, matching Slice 8, parse worker và rerank provider. Dù chưa phải full E2E suite, bộ test này phản ánh các rủi ro chính của MVP.

Table 4.4: Các nhóm test hiện có trong backend

File test	Ý nghĩa
<code>test_app_surface.py</code>	Kiểm tra app surface và route baseline.
<code>test_auth.py</code>	Kiểm tra register/login/logout/JWT behavior.
<code>test_authorization.py</code>	Kiểm tra role/ownership/disabled user guard.
<code>test_contract.py</code>	Kiểm tra contract expectations ở API layer.
<code>test_documents.py</code>	Kiểm tra document upload, parse job và storage behavior.
<code>test_embedding_provider.py</code>	Kiểm tra local/OpenAI embedding provider boundary.
<code>test_llm_parser.py</code>	Kiểm tra parser adapter và enum validation.
<code>test_matching_helpers.py</code>	Kiểm tra hard filters/scoring/reasoning helpers.
<code>test_matching_slice8.py</code>	Kiểm tra matching hardening và rerank fallback.
<code>test_parse_worker.py</code>	Kiểm tra parse worker pipeline foundation.
<code>test_rerank_provider.py</code>	Kiểm tra local reranker provider behavior.

Theo rule repository, runtime/API verification khi sửa backend phải chạy qua Docker Compose. Tuy nhiên trong phạm vi báo cáo này, không có code backend bị thay đổi; vì vậy verification chính là build LaTeX, page count và content QA. Các test backend được báo cáo như bằng chứng coverage hiện có, không chạy lại như một phần bắt buộc của artifact LaTeX.

4.9 Đối chiếu hiện trạng với target contract

Implementation matrix chia status thành **Aligned**, **Partial**, **Drift** và **Runtime-only**. Tại thời điểm tài liệu hiện tại, runtime đã cover toàn bộ named endpoints trong target contract surface ở mức path-level; không có endpoint hoàn toàn missing theo matrix. Tuy vậy còn nhiều coverage gaps server-side hoặc response-shape.

Table 4.5: Tóm tắt đối chiếu hiện trạng triển khai

Khu vực	Đã aligned/dã làm	Partial/gap còn lại
Auth/session	JWT có exp, expired token bị reject, /api/me trả bootstrap data.	Revocation list vẫn future work.
Ownership/authorization	Role-specific profile endpoint, organization membership check, matching anchor ownership.	Privacy DTO cho recruiter-visible resume detail cần hardening.
Documents/upload	Multipart upload, storage adapter, parse job creation, audit retry.	Parse review payload còn thin so với target.
Embeddings/search	Provider boundary, local fallback, semantic search query embedding, 503 envelope.	Backfill/re-embedding operation vẫn future work.
Matching	Hard filters, deterministic scoring, local rerank fallback, runtime metrics.	Quality benchmark cần labeled data.
Applications/invites	Status graph, duplicate invite guard, accept invite creates/returns application.	DTO linked display summaries còn partial ở một số list/detail.
Notifications/email	In-app rows, read/read-all, email delivery status queued.	Email adapter chưa gọi provider thực.
Admin monitoring	Nhiều endpoint read-heavy và filter đã aligned.	Audit consistency cho mọi admin read cần quyết định chính sách.

Nhận xét quan trọng là hệ thống đã đi xa hơn prototype matching đơn lẻ. Nó có route surface production, schema migration, upload foundation, provider boundary và lifecycle modules. Nhưng để gọi là production-ready hoàn toàn, các phần provider thực, parse review payload, email delivery, frontend integration và benchmark vẫn cần hoàn thiện.

4.10 Đánh giá theo yêu cầu sản phẩm

4.10.1 Điểm đáp ứng tốt

Thứ nhất, shared-pool marketplace được thể hiện rõ bằng visibility rule: active resumes và published jobs. Cả schema, HLD và runtime docs đều nhất quán với quy tắc này. Thứ hai, matching không tạo application; apply hoặc accepted invite mới tạo application. Điều này bảo vệ ranh giới giữa recommendation và business action. Thứ ba, API namespaces production đã thay thế prototype routes, giúp frontend tương lai tích hợp theo `/api/*`. Thứ tư, provider boundaries cho LLM, embedding, storage và rerank giúp hệ thống chạy được local đồng thời sẵn sàng dùng service bên ngoài.

4.10.2 Điểm còn partial

Parse review payload là gap lớn nhất đối với trải nghiệm upload-parse-review. Target yêu cầu parse job detail expose normalized fields, hard-filter fields, extracted text/reference, parser metadata, embedding metadata, warnings và linked target ID. Runtime hiện đã có parse jobs nhưng response còn thin. Nếu frontend review screen cần highlight field confidence hoặc cho user sửa từng field, API phải mở rộng.

Email delivery cũng còn partial. Business notifications đã có in-app rows, nhưng email provider chưa được gọi. Nếu sản phẩm cần gửi email invite/application status, hệ thống cần email adapter, retry policy, delivery failure logging và possibly dead-letter/ops monitoring.

Application/Invite DTO còn thiếu linked display summaries ở một số list/detail. Đây là gap frontend integration: nếu table cần job title, organization name hoặc resume title mà response chưa denormalize, frontend sẽ phải fetch thêm nhiều endpoint, tăng latency và complexity.

4.10.3 Rủi ro kỹ thuật

Rủi ro thứ nhất là chất lượng parser/embedding khi chuyển từ local fallback sang provider thật. Local deterministic parser ổn cho dev/test nhưng không đại diện hoàn toàn cho CV/JD thực tế. OpenAI-compatible parser cần kiểm soát enum validation và failure modes. Rủi ro thứ hai là thiếu labeled evaluation data nên không thể kết luận production matching quality. Tài liệu yêu cầu đã nói rõ production quality benchmarking conclusions không thuộc MVP nếu chưa có labeled evaluation data [1]. Rủi ro thứ ba là frontend chưa active runtime, nên các giả định về response shape cần được khóa qua OpenAPI trước khi triển khai UI.

4.11 Kịch bản thực nghiệm đề xuất

Vì repository hiện có unit tests và docs nhưng chưa có labeled benchmark dataset, phần thực nghiệm phù hợp nhất là smoke-contract và scenario-based verification. Các kịch bản đề xuất gồm:

1. Auth smoke: register candidate/recruiter, login, gọi `/api/me`, kiểm tra JWT exp và role bootstrap.
2. Candidate flow: create candidate profile, create draft resume, activate resume, verify resume xuất hiện trong recruiter search.
3. Recruiter flow: create organization/profile, create draft job, publish job, verify job xuất hiện trong candidate search.
4. Document flow: upload PDF, verify document row, parse job queued/processing/succeeded hoặc failed có error.
5. Matching flow: tạo một published job và một active resume pass hard filters, run matching, kiểm tra final score, component scores và reasoning.
6. Application flow: candidate apply, verify application submitted, application event, notification và audit.

7. Invite flow: recruiter invite, candidate accept, verify application created hoặc reused; reject invite không tạo application.
8. Admin flow: admin list users/documents/parse jobs/audit logs với pagination/filter.

Các kịch bản này bám sát target runtime flows, không cần dataset lớn nhưng đủ chứng minh hệ thống hoạt động xuyên suốt. Khi có labeled data, có thể bổ sung evaluation metrics cho ranking như precision@k, recall@k, nDCG hoặc agreement với reviewer. Tuy nhiên nếu chưa có nhãn, báo cáo chỉ nên nói về correctness/smoke/contract, không nên claim chất lượng matching ở mức production.

4.12 Nhận xét chương

Chương này cho thấy JobConnect đã có nền tảng implementation tương đối đầy đủ cho backend MVP: FastAPI route surface, PostgreSQL schema, documents/upload foundation, matching helpers, lifecycle modules, notifications và admin monitoring. Các dependency và file test cụ thể xác nhận hệ thống không chỉ là thiết kế trên giấy. Đồng thời, implementation matrix chỉ ra rõ các phần còn partial, đặc biệt parse review payload, email provider, frontend display DTOs và benchmark data. Đây là cơ sở để phần Kết luận đưa ra hướng phát triển thực tế thay vì liệt kê chung chung.

Kết luận

Kết quả đạt được

Khóa luận đã trình bày quá trình phân tích và thiết kế JobConnect MVP như một hệ thống tuyển dụng production-oriented, không còn là prototype matching riêng lẻ. Hệ thống được mô hình hóa thành shared-pool marketplace với ba vai trò candidate, recruiter và admin. Candidate upload CV, review parsed resume và activate resume; recruiter tạo hoặc upload JD, review parsed job và publish job; cả hai phía có thể chạy matching hai chiều; application chỉ được tạo bởi candidate apply hoặc accepted invite; notifications và audit logs ghi nhận các sự kiện nghiệp vụ quan trọng.

Về mặt kỹ thuật, khóa luận đã làm rõ kiến trúc backend FastAPI/Python của dự án. Các module runtime được tổ chức dưới `backend/src/jobconnect` theo app, core, common, integrations và domain modules. Báo cáo đã chỉ ra cụ thể các dependency đang dùng: FastAPI 0.115.6, Pydantic 2.11.7, psycopg 3.2.3, numpy 1.26.4, python-multipart, pypdf, httpx và sentence-transformers. Thiết kế data storage dùng PostgreSQL và pgvector, với schema entities rõ ràng cho users, profiles, organizations, resumes, jobs, documents, parse jobs, embeddings, applications, invites, notifications và audit logs.

Khóa luận cũng đã trình bày pipeline CV/JD từ upload file đến structured data và embeddings. Pipeline gồm object storage, uploaded document metadata, parse job lifecycle, extraction, preprocessing, Unicode NFC normalization, skill normalization, LLM/local structured parser, user review, embedding generation và visibility activation/publishing. Các provider boundaries cho storage, LLM parser, embedding provider

và reranker giúp hệ thống vận hành local nhưng vẫn sẵn sàng tích hợp provider thật.

Về matching, khóa luận đã mô tả đầy đủ hard filters, công thức scoring, semantic retrieval, optional rerank, deterministic tie-break và reasoning có căn cứ. Công thức `final_score` không được xem là chi tiết phụ, mà là core của hệ thống giải thích: title similarity, skill similarity, requirement-experience similarity và requirement-summary similarity đều có trọng số rõ. Cách thiết kế này giúp hệ thống tránh black-box matching và hỗ trợ người dùng hiểu vì sao một kết quả được xếp hạng cao.

Hạn chế

Hạn chế thứ nhất là parse review payload trong runtime hiện vẫn chưa đầy đủ so với target contract. Upload document và parse job foundation đã có, nhưng parse job detail cần mở rộng để trả normalized fields, hard-filter fields, extracted text/reference, parser metadata, embedding metadata, warnings và linked target IDs cho frontend review page. Nếu thiếu payload này, user khó review và sửa dữ liệu trước khi activate resume hoặc publish job.

Hạn chế thứ hai là email provider chưa hoàn chỉnh. Notifications in-app đã có, và email delivery status có thể ở trạng thái queued, nhưng provider gửi email thật và failure handling/retry vẫn là future work. Trong sản phẩm tuyển dụng, email invite, application status change và parse failure notification là các touchpoint quan trọng; do đó email adapter cần được triển khai trước khi production.

Hạn chế thứ ba là frontend runtime chưa active. Repository hiện ghi rõ frontend runtime code chưa phải source of truth; các tài liệu frontend đang là planning notes. Vì vậy báo cáo chưa thể đánh giá trải nghiệm người dùng thực tế trên màn hình Job Market, Talent Market, upload-parse-review hoặc application management. Khi frontend được triển khai, API response DTO cần được kiểm tra lại để tránh per-row fetch và để đảm bảo privacy của candidate contact data.

Hạn chế thứ tư là chưa có labeled evaluation dataset cho matching. Vì vậy không thể kết luận production quality theo precision, recall, nDCG hoặc human relevance agreement. Hiện có thể đánh giá correctness, contract, hard-filter behavior và deterministic

score, nhưng chất lượng ranking thực tế cần dữ liệu nhãn từ chuyên gia hoặc lịch sử tuyển dụng.

Hướng phát triển

Hướng phát triển đầu tiên là hoàn thiện parse worker và parse review API. Cần đảm bảo parse job detail trả đủ dữ liệu cho frontend review form, bao gồm parsed normalized fields, hard-filter fields, warnings, parser version, embedding version và extracted text reference. Sau khi user save corrections, hệ thống phải refresh embeddings cho các field ảnh hưởng đến search/matching.

Hướng thứ hai là triển khai email adapter và retry policy. Email failure không được rollback business transaction, nhưng phải được log rõ và có cơ chế retry hoặc operational review. Adapter nên có interface tương tự LLM/embedding/storage để môi trường local có thể dùng fake/local provider, còn production dùng provider thật.

Hướng thứ ba là hoàn thiện frontend production integration. Các endpoint job list/search/semantic, application list/detail và invite list/detail cần trả linked display summaries đủ cho UI. Candidate contact data phải được kiểm soát theo role và flow. OpenAPI contract nên là nguồn chính để generate hoặc validate client.

Hướng thứ tư là xây dựng bộ dữ liệu benchmark có nhãn cho matching. Mỗi cặp JD-CV nên có đánh giá từ recruiter hoặc chuyên gia theo mức phù hợp. Từ đó có thể đo precision@k, recall@k, nDCG và so sánh deterministic formula, local reranker, provider embedding và các biến thể scoring. Khi có dữ liệu này, hệ thống mới có cơ sở để thay đổi trọng số `final_score` hoặc thêm bonus/penalty đo được.

Hướng cuối cùng là tăng cường observability và admin monitoring. Các tín hiệu parse queue, matching latency, provider failure, notification delivery và audit consistency cần được tổng hợp thành dashboard hoặc report vận hành. Điều này giúp hệ thống không chỉ chạy đúng trong test mà còn có khả năng vận hành, debug và cải tiến liên tục khi dữ liệu thực tế tăng lên.

Kết luận chung

JobConnect MVP cho thấy một hướng tiếp cận thực tế để xây dựng hệ thống tuyển dụng có AI hỗ trợ nhưng vẫn kiểm soát được bằng rule, schema và audit. Thay vì để AI tự quyết định phù hợp hay không phù hợp, hệ thống đặt AI vào các boundary cụ thể: parse structured fields, tạo embedding, optional reasoning. Các quyết định nghiệp vụ quan trọng như visibility, hard filters, application creation, invite acceptance và status transition được điều khiển bằng rule rõ ràng. Đây là nền tảng phù hợp để phát triển tiếp thành một sản phẩm tuyển dụng có khả năng mở rộng và giải thích được.

Bibliography

- [1] JobConnect repository, “REQUIREMENTS.md – Production MVP Recruiting Marketplace”, tài liệu yêu cầu sản phẩm nội bộ.
- [2] JobConnect repository, “Production HLD: Overview And Problem”, tài liệu kiến trúc mức cao.
- [3] JobConnect repository, “Production HLD: Architecture Overview”, tài liệu kiến trúc backend.
- [4] JobConnect repository, “Production HLD: Ingestion And Normalization”, tài liệu pipeline xử lý CV/JD.
- [5] JobConnect repository, “Production HLD: Data And Storage”, tài liệu thiết kế dữ liệu và lưu trữ.
- [6] JobConnect repository, “Production HLD: Matching And Search Pipeline”, tài liệu thiết kế matching và search.
- [7] JobConnect repository, “Production HLD: API And Runtime Flows”, tài liệu luồng runtime và API.
- [8] JobConnect repository, “Production HLD: Security, Notifications, And Audit”, tài liệu bảo mật, notification và audit.
- [9] JobConnect repository, “Production LLD: Database Schema”, tài liệu thiết kế schema PostgreSQL và pgvector.

- [10] JobConnect repository, “Production LLD: API Contract”, tài liệu hợp đồng API production.
- [11] JobConnect repository, “Current API Implementation Matrix”, tài liệu đối chiếu hiện trạng triển khai với target contract.
- [12] JobConnect repository, “README.md”, tài liệu tổng quan dự án, setup và layout backend.
- [13] Greenhouse, “Greenhouse platform”, <https://www.greenhouse.com/platform>.
- [14] Lever, “LeverTRM ATS + CRM”, <https://www.lever.co/lever-trm>.
- [15] Workday, “Workday Talent Acquisition”, <https://www.workday.com/en-us/products/talent-management/talent-acquisition.html>.

APPENDIX A

Phụ lục API, schema và traceability

A.1 Tóm tắt API contract

Phụ lục này gom các bảng kỹ thuật để khi đọc báo cáo có thể tra nhanh endpoint, entity, enum và nguồn tài liệu. Nội dung được tổng hợp từ `docs/backend/LLD/40-api-contract.md`, `docs/backend/LLD/30-database-schema.md`, `docs/backend/LLD/50-current-state.md` và các HLD liên quan.

Table A.1: API endpoint summary theo nhóm chức năng

Nhóm	Endpoint	Vai trò	Ghi chú
Auth	POST /api/auth/register	Public	Tạo user, hash password, issue JWT.
Auth	POST /api/auth/login	Public	Verify password, block disabled user, issue JWT.
Auth	POST /api/auth/logout	Authenticated	Stateless logout, client discard token.
User	GET /api/me	Authenticated	User summary + role-specific bootstrap.
Candidate profile	GET /api/candidate/profile	Candidate	Đọc profile hiện tại.
Candidate profile	PUT /api/candidate/profile	Candidate	Upsert candidate profile.
Recruiter profile	GET /api/recruiter/profile	Recruiter	Đọc recruiter profile.
Recruiter profile	PUT /api/recruiter/profile	Recruiter	Upsert profile sau organization check.

Nhóm	Endpoint	Vai trò	Ghi chú
Organizations	GET /api/organizations	Authenticated	List/search organization với q, limit, offset.
Organizations	POST /api/organizations	Recruiter/Admin runtime	Tạo organization, audit event.
Organizations	GET /api/organizations/{id}	Authenticated	Detail organization.
Organizations	PATCH /api/organizations/{id}	Recruiter/Admin	Update organization với membership check.
Resumes	GET /api/candidate/resumes	Candidate/Admin	List resumes theo owner hoặc all admin.
Resumes	POST /api/candidate/resumes	Candidate	Create draft resume, upsert embeddings.
Resumes	GET /api/candidate/resumes/search	Recruiter/Admin	Search active resumes theo q/filters.
Resumes	POST /api/candidate/resumes/semantic-search	Recruiter/Admin	Semantic search resume relevance.
Resumes	GET /api/candidate/resumes/{id}	Candidate/Recruiter/Admin	Owner/all/active visibility.
Resumes	PATCH /api/candidate/resumes/{id}	Candidate	Partial update và refresh embeddings.
Resumes	POST /api/candidate/resumes/{id}/activate	Candidate	Draft/archived -> active; audit event.
Resumes	POST /api/candidate/resumes/{id}/archive	Candidate	Draft/active -> archived; audit event.
Jobs	GET /api/jobs	Authenticated active	List jobs với filters.
Jobs	POST /api/jobs	Recruiter/Admin runtime	Create draft job, upsert embeddings.
Jobs	GET /api/jobs/search	Authenticated active	Search title/skills/organization name.
Jobs	POST /api/jobs/semantic-search	Authenticated active	Semantic job search theo requirement embeddings.

Nhóm	Endpoint	Vai trò	Ghi chú
Jobs	GET /api/jobs/{id}	Authenticated active	Candidate sees published, recruiter owns, admin all.
Jobs	PATCH /api/jobs/{id}	Recruiter/Admin	Partial update và refresh embeddings.
Jobs	POST /api/jobs/{id}/publish	Recruiter/Admin	Draft -> published, set published_at .
Jobs	POST /api/jobs/{id}/close	Recruiter/Admin	Draft/published -> closed.
Documents	POST /api/documents	Authenticated active	Multipart upload, storage write, parse job queued.
Documents	GET /api/documents	Authenticated active	List visible documents.
Documents	GET /api/documents/{id}	Owner/Admin	Document detail + parse jobs.
Documents	GET /api/documents/{id}/download-url	Owner/Admin	Storage adapter URL.
Documents	GET /api/documents/{id}/parse-jobs/{pid}	Owner/Admin	Parse job detail.
Documents	POST /api/documents/{id}/parse-jobs	Owner/Admin	Retry parse job, audit event.
Matching	POST /api/matching/jobs/{id}/run	Recruiter/Admin	Published job anchor -> active resumes.
Matching	POST /api/matching/resumes/{id}/run	Candidate/Admin	Active resume anchor -> published jobs.
Applications	GET /api/applications	Candidate/Recruiter/Admin	Role-scoped application listing.
Applications	POST /api/applications	Candidate	Candidate apply, event, notification, audit.
Applications	GET /api/applications/{id}	Candidate/Recruiter/Admin	Detail + event history.
Applications	POST /api/applications/{id}/status	Authenticated active	Role-specific transition graph.
Invites	GET /api/invites	Candidate/Recruiter/Admin	Role-scoped invite listing.

Nhóm	Endpoint	Vai trò	Ghi chú
Invites	POST /api/invites	Recruiter	Create pending invite, duplicate guard.
Invites	GET /api/invites/{id}	Candidate/Recruiter/Admin	Detail invite.
Invites	POST /api/invites/{id}/accept	Candidate	Accept and create/return application.
Invites	POST /api/invites/{id}/reject	Candidate	Reject, no application.
Notifications	GET /api/notifications	Authenticated active	List notifications.
Notifications	POST /api/notifications/{id}/read	Owner	Mark one notification read.
Notifications	POST /api/notifications/read-all	Authenticated active	Mark unread notifications read, return count.
Admin	GET /api/admin/users	Admin	User monitoring.
Admin	GET /api/admin/users/{id}	Admin	User detail + profile context + ops summary.
Admin	GET /api/admin/documents	Admin	Document monitoring filters.
Admin	GET /api/admin/parse-jobs	Admin	Parse job monitoring filters.
Admin	GET /api/admin/applications	Admin	Application monitoring filters.
Admin	GET /api/admin/invites	Admin	Invite monitoring filters.
Admin	GET /api/admin/notifications	Admin	Notification monitoring filters.
Admin	GET /api/admin/audit-logs	Admin	Audit log filters.

A.2 Entity summary

Table A.2: Tóm tắt entity và field chính

Entity	Field chính	Invariant
users	user_id, email, password_hash, role, status	Email unique; password chỉ lưu hash; disabled user bị chặn active actions.
candidate_profiles	user_id, full_name, current_location, headline	Một row cho candidate; total experience chỉ là display metadata.
recruiter_profiles	user_id, organization_id, full_name, title	Một row cho recruiter; organization kiểm soát recruiting context.
organizations	organization_id, name, slug, logo_url	Có predefined Independent organization cho lựa chọn “Khác”.
candidate_resumes	resume_id, candidate_user_id, title, skills, status	Chỉ active vào public pool; is_primary không điều khiển visibility.
job_posts	job_id, organization_id, recruiter_user_id, title, status	Chỉ published vào public pool; published_at set khi publish.
uploaded_documents	document_id, owner_user_id, document_type, object_key, mime_type	File gốc nằm trong object storage; DB lưu meta-data/reference.
parse_jobs	parse_job_id, document_id, status, parser_version, error_code	Failed parse không xóa file; succeeded phải link đúng target entity.
candidate_resume_embeddings	resume_id, emb_title, emb_skills, emb_summary, emb_experience	Vector 384 chiều; missing vector score component 0.
job_post_embeddings	job_id, emb_title, emb_skills, emb_requirement	Có embedding_version để backfill.
applications	application_id, job_id, resume_id, status	Unique (job_id, resume_id); matching không tạo application.
application_events	application_id, from_status, to_status, actor_user_id	Append-only status history.

Entity	Field chính	Invariant
recruiter_invites	invite_id, job_id, resume_id, status	Pending invite không tạo application; accepted invite tạo/reuse application.
notifications	notification_id, recipient_user_id, type, status	Email failure không rollback business transaction.
audit_logs	actor_user_id, event_type, target_type, metadata	Business event trail cho lifecycle actions.

A.3 Matching formula và hard filter details

Table A.3: Score component trong matching

Component	Weight	Ý nghĩa
title_sim	0.35	Mức tương đồng giữa title/resume title hoặc định danh nghề nghiệp.
skills_sim	0.35	Kết hợp semantic skills và exact overlap ratio.
req_exp_sim	0.20	Tương đồng giữa yêu cầu JD và kinh nghiệm CV.
req_summary_sim	0.10	Tương đồng giữa requirement và summary.
bonus_exact_skill	0 hiện tại	Dự phòng cho measured rule sau này.
penalty_missing_required	0 hiện tại	Dự phòng; hard filters đã loại pair thiếu điều kiện.

Table A.4: Reasoning content tối thiểu

Nội dung	Mục đích
Strongest components	Cho người dùng biết score cao nhờ title, skills, experience hay summary.
Exact skill overlap	Giải thích kỹ năng trùng khớp cụ thể giữa CV và JD.

Nội dung	Mục đích
Hard-filter pass notes	Xác nhận seniority, location, education, certifications có pass không.
Missing embedding notes	Nếu vector field thiếu và component bị 0, reasoning phải nêu rõ.
Warnings metadata	Nêu rerank fallback hoặc provider unavailable nếu có.

A.4 Source traceability

Table A.5: Traceability từ nội dung báo cáo về source docs

Nội dung báo cáo	Nguồn chính	Ghi chú sử dụng
Product goal và user roles	docs/REQUIREMENTS.md	Candidate, recruiter, admin; shared-pool marketplace.
MVP scope/non-goals	docs/REQUIREMENTS.md	Tránh claim tenant isolation, billing, scheduling, benchmark quality.
Canonical enums	docs/REQUIREMENTS.md, API LLD	Dùng cho API/persistence validation.
Ingestion pipeline	HLD ingestion	Upload -> storage -> parse -> review -> embedding.
LLM parser boundary	HLD ingestion	integrations/llm/base.py, local/openai adapters.
Embedding provider	HLD matching	Local hash/OpenAI text-embedding-3-small, 384 dimensions.
Data model	HLD data, DB LLD	Entity tables, constraints, indexes, invariants.
Matching hard filters	Requirements, HLD matching	Job type, location, seniority, education, certifications.
Scoring formula	Requirements, HLD matching	final_score và skills_sim.
Runtime flows	HLD API/runtime	Candidate flow, recruiter flow, matching flow, application flow.
Security/privacy	HLD security	JWT, disabled user, role/ownership, contact data privacy.

Nội dung báo cáo	Nguồn chính	Ghi chú sử dụng
Notifications/audit	Requirements, HLD security	Notification types, email failure, audit events.
API namespaces	Requirements, API LLD	Production / <code>api</code> /*, no prototype aliases.
Implementation status	Current API Implementation Matrix	Aligned/partial/gap analysis.
Backend layout	README, codemap, file tree	App/core/common/integrations/modules.
Dependencies	backend/requirements.txt	Versioned runtime packages.
Report structure	report/knowledge/01-04	Bố cục khóa luận, giọng viết, no DOCX reopen by default.

A.5 Checklist dùng lại khi tiếp tục phát triển báo cáo

Table A.6: Checklist kiểm tra nội dung báo cáo

STT	Tiêu chí
1	Mở đầu và Kết luận không đánh số chương.
2	Bốn chương nội dung tương ứng đúng vai trò: bối cảnh, kỹ thuật, thiết kế, thực nghiệm.
3	Mọi mô tả công nghệ phải gắn với file/version/module cụ thể khi có trong repo.
4	Không dùng mô tả công nghệ dạng chung chung; mọi công nghệ phải gắn với version, module hoặc file nguồn khi có dữ liệu.
5	Matching phải có hard filters, công thức score, tie-break và reasoning.
6	Data model phải nêu rõ public pool visibility: active resumes, published jobs.
7	API phải dùng namespace production / <code>api</code> /*, không dùng prototype route.
8	Các claim về implementation status phải bám implementation matrix.
9	Những gì chưa biết hoặc chưa có trong docs phải để placeholder hoặc nêu là future work.

STT	Tiêu chí
10	Nếu DOCX nguồn thay đổi, regenerate Knowledge Pack trước khi viết tiếp.